

claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-ab8ad46e9cb84dca5.jsonl`  
- SHA-256: `76a9fa63614cec2020d834a5896f9ed77b5a736888c4ab91f16f67a6d4e456c3`  
- Source modified: `2026-07-07T15:13:06+00:00`  
- Imported at: `2026-07-09T08:32:24+00:00`  
- project: `wf_a8f43370-4cb`  
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`
```

Transcript

1. user (2026-07-07T15:07:26.829Z)

You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic "WASB weights not found at /opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):

```
- BACKEND (repo undefined, branch undefined, PR #undefined):  
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,  
    fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote  
  the  
    choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-  
  backed  
    AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-  
  process as  
    before). Plus 4 tests in tests/test_api_cloud_dispatch.py.  
- SPA (repo undefined, branch undefined, PR #undefined):  
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when  
  capabilities.deployment.compute_target=="cloud_run" (else keeps the free "local" default).  
Plus 1 test.
```

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:

```
- undefined/backend.diff (backend commit diff)  
- undefined/orchestration_new.py (FULL post-change backend/orchestration.py)  
- undefined/test_api_cloud_dispatch_new.py (FULL post-change tests)  
- undefined/spa.diff (SPA commit diff)  
- undefined/IngestPanel_new.tsx (FULL post-change IngestPanel.tsx)
```

You MAY read other repo files for ground truth, e.g. under undefined:

```
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;  
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;  
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);  
and under undefined: web/src/api/types.ts, web/src/api/client.ts,  
web/src/components/IngestPanel.test.tsx.
```

You may also run: `gh pr diff undefined -R Khelsutra/badminton-highlight-indexer` and `gh pr diff undefined -R avidullu/khelsutra`.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete config/inputs

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.

If nothing real exists in your dimension, return an empty findings array.

=== YOUR DIMENSION: test-adequacy ===

Are the NEW tests correct and SUFFICIENT, and do they actually exercise the guardrail (not tautologies)? Read test_api_cloud_dispatch_new.py and IngestPanel.test.tsx (in undefined). Check: do the backend tests assert the exact failure message + provenance (path=not_run, dispatched=False) and that NO dispatch happened? Is the escape-hatch test real (weights present ?

in_process)? Gaps: is fusion_hybrid / tracknet_hybrid covered at all? Is the ENV-WIRED cloud_available

case (compute_target="" + env) covered? For the SPA, is there a test proving a local appliance (cloud_available=true but compute_target?cloud_run) STILL defaults to local (guarding against over-defaulting to cloud)? Does the new SPA test truly assert compute:"cloud" is sent and "local"

is never sent by default? Flag missing high-value tests as findings.

2. assistant (2026-07-07T15:07:32.617Z)

I'll start by exploring the environment to locate the snapshot files and understand the repo structure.

3. user (2026-07-07T15:07:35.078Z)

=== CWD ===

/c/Users/avidu/Projects/khelsutra-guru/khelsutra

=== CWD contents ===

total 62

```
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 22 11:12 .claude
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:33 .git
-rw-r--r-- 1 avidu 197609 297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609 323 Jun 22 14:57 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 docs
drwxr-xr-x 1 avidu 197609    0 Jun 21 23:55 locales
-rw-r--r-- 1 avidu 197609 9773 Jul  2 21:13 NEXT_STEPS.md
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:38 node_modules
-rw-r--r-- 1 avidu 197609 1469 Jun 23 00:19 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 23 00:19 site
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:45 web
```

=== parent ===

total 108

```
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:24 ..
drwxr-xr-x 1 avidu 197609    0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609 905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:34 bhi-wt-worker-reval
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609    0 Jul  5 18:47 khelsutra-screenshot
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport
```

4. user (2026-07-07T15:07:41.645Z)

```

=== find snapshots ===
=== also temp ===
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/backend.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/IngestPanel_new.tsx
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/spa.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr37.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr38.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr39.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr50.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr51.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr52.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr53.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr54.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr55.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr57.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr58.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/pr55.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/edc346c2-92fc-4ad1-b556-c7ab955a5613/scratchpad/pr158.diff

```

5. assistant (2026-07-07T15:07:50.460Z)

The snapshot files are in my scratchpad. Let me list the full directory and read the key files. My dimension is test-adequacy, so the test files are central.

6. user (2026-07-07T15:07:52.378Z)

```

total 156
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:35 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:31 ..
-rw-r--r-- 1 avidu 197609 11027 Jul  7 20:35 backend.diff
-rw-r--r-- 1 avidu 197609 29098 Jul  7 20:35 IngestPanel_new.tsx
-rw-r--r-- 1 avidu 197609 55735 Jul  7 20:35 orchestration_new.py
-rw-r--r-- 1 avidu 197609  5007 Jul  7 20:35 spa.diff
-rw-r--r-- 1 avidu 197609 42525 Jul  7 20:35 test_api_cloud_dispatch_new.py

```

7. assistant (2026-07-07T15:07:56.955Z)

Let me read the diffs and the test files in parallel.

8. user (2026-07-07T15:07:58.034Z)

```

1      commit 4565c7436ccf0b4e30f68f01efe6082eff371a5d
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:56:22 2026 +0530
4
5          fix(serving): fail fast when compute=local forces weightless in-process detection
6
7          A hosted cloud-serving control-plane bakes NO detector weights into its image
8          (only the worker Cloud Run Job GCSFuse-mounts them at /gcs/models/...). So a
9          POST /api/process with compute="local" for a weights-backed trajectory detector
10         (wasb_hybrid / tracknet_hybrid / fusion_hybrid) returned None from
11         _maybe_dispatch_remote and ran the in-process segmenter ON THE CONTROL-PLANE,
12         which then died deep in the healthcheck with the confusing
13         "WASB weights not found at /opt/models/WASB-SBDT/..." (observed live 2026-07-07,
14         job f29d41d0?: [COMPUTE] requested=local actual=in_process ... status=failed).
15
16         Guardrail: on a cloud-serving-capable server (_cloud_available) whose local
17         weights for the requested detector aren't readable, a forced compute="local"
18         now fails FAST with an actionable message ("this hosted server can't run
19         detection locally ... Choose 'Run in cloud'.") ? mirroring the existing
20         compute="cloud"-not-available reject ? instead of the silent in-process trap.
21         A truly-local box, or a cloud-wired box that actually has the weights on disk,
22         is unaffected (the weights-present check is the escape hatch); a weight-free
23         segmenter (motion) is out of scope. Honest provenance: path=not_run,
24         requested=local, dispatched=False.
25
26         Covered by tests/test_api_cloud_dispatch.py (fail-fast, weights-present escape
27         hatch, motion unaffected, pure-local not guardrailed).
28
29         Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
30
31     diff --git a/backend/orchestration.py b/backend/orchestration.py
32     index 5e9f510..bfe39f5 100644
33     --- a/backend/orchestration.py
34     +++ b/backend/orchestration.py
35     @@ -155,6 +155,38 @@ def _ingest_remote_segments(
36         return n
37
38
39     += Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS
40     box. A hosted
41     += cloud-serving control-plane bakes NO detector weights into its image (only the worker
42     Cloud Run Job
43     += GCSFuse-mounts them), so forcing compute="local" for one of these there can never
44     succeed ? the
45     += guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
46     deep
47     += "WASB weights not found". (Names are the registered segmenter identities; see
48     backend/pipeline/segmenters.)
49     += _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
50     "fusion_hybrid"})
51     +=
52     +=
53     += def _local_detector_weights_present(cfg, seg_name: str) -> bool:
54     +=     """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
55     present + readable
56     +=     on THIS box. Mirrors the native/WSL runners' resolution (env override ?
57     ``indexing.<det>.weights_path``)
58     +=     and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
59     that would
60     +=     otherwise fail deep in the run. Fusion follows its configured substrate (WASB
61     default / TrackNet).
62     +=     A non-weights segmenter returns True (nothing local to check)."""
63     +=     name = (seg_name or "").strip().lower()
64     +=     if name not in _WEIGHTS_BACKED_SEGMENTERS:
65     +=         return True

```

```

56     +     idx = getattr(cfg, "indexing", None)
57     +     substrate = (
58     +         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
59     +     ).strip().lower()
60     +     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
61     +         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
62     +     else: # wasb_hybrid, or fusion on the (default) WASB substrate
63     +         path = (
64     +             os.environ.get("WASB_WEIGHTS")
65     +             or getattr(getattr(idx, "wasb", None), "weights_path", "")
66     +             or ""
67     +         )
68     +     return bool(path) and os.path.exists(os.path.expanduser(path))
69     +
70     +
71     def _cloud_available(cfg) -> bool:
72         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
73
74     @@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
75         )
76         choice = ""
77         if choice == "local":
78     +         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
79     +         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
80     +         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
81     +         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail
FAST with an
82     +         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
83     +         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it
runs in-process
84     +         # below exactly as before (the weights-present check is the escape hatch).
85     +         if (
86     +             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
87     +             and _cloud_available(cfg)
88     +             and not _local_detector_weights_present(cfg, seg_name)
89     +         ):
90     +             return _failed(
91     +                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
92     +                 "live only on the cloud worker, not on the control-plane. Choose 'Run
in cloud'.",
93     +                 False,
94     +             )
95     +         return None # explicit "run on my machine" ? in-process regardless of the
server's target
96         if choice == "cloud":
97             if not _cloud_available(cfg):
98     diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
99     index 8382234..7993b7f 100644
100     --- a/tests/test_api_cloud_dispatch.py
101     +++ b/tests/test_api_cloud_dispatch.py
102     @@ -721,6 +721,114 @@ def
test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypa
103         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
104
105
106     + # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-----
107     + # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud

```

```

Run Job mounts
108     ## them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
109     ## FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
110     ## that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).
111     +
112     +
113     +def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
114     +     """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
115     +     a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
116     +     _db, _bucket, _mp = cloud_env
117     +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
118     +     m.global_config.indexing.wasb.weights_path =
"/nonexistent/wasb_badminton_best.pth.tar"
119     +     fake = _FakeCloudClient()
120     +     _inject_fake(monkeypatch, fake)
121     +
122     +     out = m._execute_processing(
123     +         m.ProcessRequest(
124     +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
125     +         )
126     +     )
127     +     assert out["status"] == "failed"
128     +     assert "can't run detection locally" in out["message"]
129     +     assert "Run in cloud" in out["message"]
130     +     assert "weights not found" not in out["message"] # NOT the cryptic deep failure
131     +     assert fake.calls == [] # never dispatched
132     +     # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
133     +     assert out["compute"]["requested"] == "local"
134     +     assert out["compute"]["dispatched"] is False
135     +     assert out["compute"]["path"] == "not_run"
136     +
137     +
138     +def test_local_on_cloud_server_with_weights_present_runs_in_process(
139     +     cloud_env, tmp_path, monkeypatch
140     +):
141     +     """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
ACTUALLY has the
142     +     detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""
143     +     _db, _bucket, _mp = cloud_env
144     +     weights = tmp_path / "wasb_badminton_best.pth.tar"
145     +     weights.write_bytes(b"WEIGHTS")
146     +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
147     +     m.global_config.indexing.wasb.weights_path = str(weights)
148     +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
149     +     fake = _FakeCloudClient()
150     +     _inject_fake(monkeypatch, fake)
151     +
152     +     out = m._execute_processing(
153     +         m.ProcessRequest(
154     +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
155     +         )
156     +     )
157     +     assert out["status"] == "success"
158     +     assert fake.calls == [] # in-process, not dispatched
159     +     assert out["compute"]["path"] == "in_process"
160     +
161     +
162     +def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
163     +     cloud_env, monkeypatch

```

```

164     +):
165     +     """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
(motion) with
166     +     compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
167     +     _db, _bucket, _mp = cloud_env
168     +     m.global_config.indexing.wasb.weights_path =
"/nonexistent/should_not_matter.pth.tar"
169     +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
170     +     fake = _FakeCloudClient()
171     +     _inject_fake(monkeypatch, fake)
172     +
173     +     out = m._execute_processing(
174     +         m.ProcessRequest(
175     +             video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
176     +         )
177     +     )
178     +     assert out["status"] == "success"
179     +     assert fake.calls == []
180     +     assert out["compute"]["path"] == "in_process"
181     +
182     +
183     +def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
184     +     tmp_path, monkeypatch
185     +):
186     +     """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
187     +     weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
188     +     'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
189     +     db = Database(str(tmp_path / "t.db")) # noqa: F841
190     +     monkeypatch.setattr(m, "db_instance", db)
191     +     cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa:
F841
192     +     monkeypatch.setattr(m, "global_config", cfg)
193     +     monkeypatch.setattr(
194     +         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
195     +     )
196     +     monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
197     +     monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
198     +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
199     +     cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
200     +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
201     +     video = tmp_path / "v.mp4"
202     +     video.write_bytes(b"hashable-bytes")
203     +
204     +     out = m._execute_processing(
205     +         m.ProcessRequest(
206     +             video_path=str(video), segmenter="wasb_hybrid", compute="local"
207     +         )
208     +     )
209     +     # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
210     +     assert out["status"] == "success"
211     +     assert out["compute"]["path"] == "in_process"
212     +
213     +
214     + # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
215
216
217

```

```

1      commit 1470e2acdea4f2b478476150da61e844dce0a8e9
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:57:35 2026 +0530
4
5          fix(ingest): default the compute picker to CLOUD on a hosted control-plane
6
7      On a hosted cloud-serving control-plane (capabilities.deployment.compute_target
8      == "cloud_run") the engine bakes NO detector weights, so in-process LOCAL
9      detection always fails ("WASB weights not found"). The picker still defaulted to
10     LOCAL and sent compute="local", which the engine ran on the weightless
11     control-plane -> a confusing failure.
12
13     Default the picker to CLOUD when the server advertises compute_target=cloud_run
14     (cloud_available too), so the SPA never sends compute=local by default on the
15     hosted build. The free-LOCAL default is kept for a local appliance that merely
16     also offers cloud (compute_target != cloud_run). The cloud cost-acknowledgement
17     gate is unchanged (no paid dispatch without consent).
18
19     Pairs with the engine-side fast-fail guardrail (badminton-highlight-indexer
20     _maybe_dispatch_remote) for anyone who still explicitly picks LOCAL on the
21     hosted build. Covered by an IngestPanel test.
22
23     Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
24
25     diff --git a/web/src/components/IngestPanel.test.tsx
26     b/web/src/components/IngestPanel.test.tsx
27     index 685aa1b..b39e4f1 100644
28     --- a/web/src/components/IngestPanel.test.tsx
29     +++ b/web/src/components/IngestPanel.test.tsx
30     @@ -185,6 +185,30 @@ describe("IngestPanel", () => {
31         expect(bodies[0].compute).toBe("local");
32     });
33     + it("defaults to CLOUD (never sends compute=local) on a hosted cloud_run control-
34     plane", async () => {
35         + // A hosted control-plane bakes no detector weights, so LOCAL can't run there ? the
36         picker must
37         + // default to CLOUD. The cost gate still applies (accept it, then the sole submit
38         is compute=cloud).
39         + const bodies: Record<string, unknown>[] = [];
40         + vi.stubGlobal(
41             "fetch",
42             fetchAll({
43                 caps: { deployment: { cloud_available: true, compute_target: "cloud_run" } },
44                 onProcessBody: (b) => bodies.push(b),
45                 jobs: [{ job_id: "j", status: "done", video_id: "vidH", result: { status:
46         "success", video_id: "vidH" } } ]},
47             ),
48             );
49         + render(<IngestPanel onLoaded={vi.fn()} />);
50         + await screen.findByTestId("compute-picker");
51         + // Defaulted to cloud ? the cost note + ack are shown up front (LOCAL would hide
52         them).
53         + expect(screen.getByTestId("compute-cost")).toBeTruthy();
54         + typeUri("gs://bucket/match.mp4");
55         + fireEvent.click(screen.getByRole("checkbox")); // accept the cost gate
56         + clickProcess();
57         + await act(async () => {});
58         + expect(bodies).toHaveLength(1);
59         + expect(bodies[0].compute).toBe("cloud"); // never "local" by default on the hosted
60         build
61         + });
62         +
63         it("shows a pasted host-path basename while processing and clears it on cancel",
64         async () => {

```

```

58     vi.useFakeTimers();
59     vi.stubGlobal(
60     diff --git a/web/src/components/IngestPanel.tsx b/web/src/components/IngestPanel.tsx
61     index 38ca3a9..edfd5d9 100644
62     --- a/web/src/components/IngestPanel.tsx
63     +++ b/web/src/components/IngestPanel.tsx
64     @@ -81,7 +81,9 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
65
66     // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to
a cloud GPU
67     // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
68     - // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before
it can run.
69     + // (free, no surprise cost) EXCEPT on a hosted cloud-serving control-plane
(compute_target=cloud_run):
70     + // there the control-plane bakes no detector weights, so LOCAL can never run ?
default CLOUD instead
71     + // (set from capabilities below), which requires an explicit cost acknowledgement
before it can run.
72     const [cloudAvailable, setCloudAvailable] = useState(false);
73     const [compute, setCompute] = useState<Compute>("local");
74     const [cloudAck, setCloudAck] = useState(false);
75     @@ -101,6 +103,13 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
76     .then(c => {
77         if (!alive) return;
78         setCloudAvailable(c.deployment?.cloud_available === true);
79     +     // Hosted cloud-serving control-plane (compute_target=cloud_run) bakes NO
detector weights, so
80     +     // in-process LOCAL detection there always fails ("weights not found") ?
default the picker to
81     +     // CLOUD so the SPA never sends compute=local by default on the hosted build. A
local appliance
82     +     // that merely also offers cloud (compute_target?cloud_run) keeps the free
LOCAL default.
83     +     if (c.deployment?.cloud_available === true && c.deployment?.compute_target ===
"cloud_run") {
84     +         setCompute("cloud");
85     +     }
86     setBackends(c.storage?.backends ?? []);
87     setMaxUploadBytes(
88     typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb
* 1024 * 1024 : undefined,
89

```

10. assistant (2026-07-07T15:08:05.977Z)

Let me read the full backend test file and the SPA test file in the repo.

11. user (2026-07-07T15:08:08.936Z)

```

1     ""API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3     Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4     intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5     GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6     done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7     dispatches; every other target runs the in-process segmenter byte-identically.
8     ""
9
10    import csv

```

```

11     import json
12     import os
13
14     import pytest
15
16     pytest.importorskip("httpx")
17     from fastapi.testclient import TestClient
18
19     import backend.compute as compute_pkg
20     import backend.main as m
21     from backend.compute.cloud_run import CloudRunJobClient
22     from backend.config.models import AppConfig
23     from backend.infrastructure.database import Database
24     from backend.infrastructure.execution_policy import ExecutionPolicy
25     from backend.pipeline.validators.base import ValidationResult
26
27     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
28
29
30     def _pass():
31         return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
32
33
34     class _InlineExecutor:
35         def submit(self, fn, *a, **k):
36             fn(*a, **k)
37
38
39     class _FakeCloudClient(CloudRunJobClient):
40         """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41         into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
42
43         def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44             self.calls: list[list[str]] = []
45             self.video_id, self.returncode = video_id, returncode
46             self.rows = (
47                 rows
48                 if rows is not None
49                 else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50             )
51
52         def run_job(self, job_name, argv, *, region, project=None):
53             self.calls.append(list(argv))
54             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55             done_uri = argv[argv.index("--done-uri") + 1]
56             if self.returncode == 0:
57                 outputs = os.path.join(out_uri, "outputs", self.video_id)
58                 os.makedirs(outputs, exist_ok=True)
59                 with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60                     json.dump(
61                         {
62                             "video_id": self.video_id,
63                             "segment_count": len(self.rows),
64                             "status": "success",
65                         },
66                         f,
67                     )
68                 with open(
69                     os.path.join(outputs, "intervals.csv"),
70                     "w",
71                     newline="",
72                     encoding="utf-8",
73                 ) as f:

```

```

74         w = csv.writer(f)
75         w.writerow(["start", "end", "shot_count", "ending_reason"])
76         for s, e, sc, er in self.rows:
77             w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78     os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79     with open(done_uri, "w", encoding="utf-8") as f:
80         json.dump(
81             {
82                 "video_id": self.video_id,
83                 "returncode": self.returncode,
84                 "segment_count": len(self.rows),
85                 "status": "success",
86             },
87             f,
88         )
89     return "exec-1"
90
91     def cancel_execution(self, execution):
92         raise AssertionError(
93             "cancel must never fire on the happy path"
94         ) # pragma: no cover
95
96
97     @pytest.fixture
98     def cloud_env(tmp_path, monkeypatch):
99         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
100         bucket read an
101         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
102         hash+validate."""
103         db = Database(str(tmp_path / "t.db")) # noqa: F841
104         monkeypatch.setattr(m, "db_instance", db)
105         monkeypatch.setattr(m, "db_instance", db)
106         bucket = tmp_path / "bucket"
107         bucket.mkdir()
108         cfg = AppConfig.model_validate(
109             { # noqa: F841
110                 "deployment": {"compute_target": "cloud_run"},
111                 "storage": {"backend": "local", "output_root": str(bucket)},
112             }
113         )
114         monkeypatch.setattr(m, "global_config", cfg)
115         monkeypatch.setattr(m, "global_config", cfg)
116         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
117         # A real gs:// input would be fetched to hash+validate; return a local fake so that
118         # path is offline.
119         fake_local = tmp_path / "fetched.mp4"
120         fake_local.write_bytes(b"FAKEVIDEOBYTES")
121         monkeypatch.setattr(
122             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
123         )
124         monkeypatch.setattr(
125             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
126         )
127         return db, bucket, monkeypatch
128
129     def _inject_fake(monkeypatch, fake):
130         """Make backend.compute.get_compute_backend (re-imported inside
131         _maybe_dispatch_remote) build a
132         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
133         token. ``**kw``
134         forwards the per-request ``target_override`` (item 3) so the cloud-picker path
135         resolves too."""
136         real = compute_pkg.get_compute_backend
137         monkeypatch.setattr(

```

```

133         compute_pkg,
134         "get_compute_backend",
135         lambda config, **kw: real(
136             config, run_client=fake, policy=_FAST, token="tok", **kw
137         ),
138     )
139
140
141 def _meta(row):
142     md = row.get("metadata")
143     return json.loads(md) if isinstance(md, str) else (md or {})
144
145
146 def test_cloud_dispatch_ingests_segments(cloud_env):
147     db, _bucket, monkeypatch = cloud_env
148     fake = _FakeCloudClient()
149     _inject_fake(monkeypatch, fake)
150
151     out = m._execute_processing(
152         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
153     )
154
155     assert out["status"] == "success"
156     assert len(out["play_segments"]) == 2
157     rows = db.query_play_segments(out["video_id"], {})
158     assert len(rows) == 2
159     metas = [_meta(r) for r in rows]
160     assert all(md["source"] == "cloud_run" for md in metas)
161     assert {md.get("shot_count") for md in metas} == {5, 3}
162     assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
163
164
165 def test_cloud_dispatch_builds_correct_spec(cloud_env):
166     _db, bucket, monkeypatch = cloud_env
167     fake = _FakeCloudClient()
168     _inject_fake(monkeypatch, fake)
169
170     m._execute_processing(
171         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
172     )
173
174     assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
175     argv = fake.calls[0]
176     assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
177     assert argv[argv.index("--out-uri") + 1] == str(bucket)
178     assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
179     joined = " ".join(argv)
180     assert (
181         "deployment.compute_target=local_gpu" in joined
182     ) # container runs INLINE (never re-dispatch)
183     assert "indexing.detector_impl=native" in joined
184     assert (
185         "indexing.skip_ai_handoff=false" in joined
186     ) # the engine's AI-handoff stance is forwarded
187
188
189 def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
190     """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
191     it tries the
192     Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
193     is forwarded."""
194     _db, _bucket, monkeypatch = cloud_env
195     m.global_config.indexing.skip_ai_handoff = True
196     fake = _FakeCloudClient()
197     _inject_fake(monkeypatch, fake)

```

```

196     m._execute_processing(
197         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
198     )
199     assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])
200
201
202     def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
203         """The worker Job has no DEPLOYMENT_PROFILE, so it never applies the cloud-serving
204         preset ?
205         the control-plane MUST forward its resolved WASB serving knobs (decode_backend +
206         GPU-feed
207         tuning) or the worker silently reverts to the model defaults (cpu decode, batch 8),
208         making the
209         preset flip (#506/#507) a no-op on the real dispatch path. Regression guard for that
210         wiring."""
211         _db, _bucket, monkeypatch = cloud_env
212         m.global_config.indexing.wasb.decode_backend = "nvdec_resident"
213         m.global_config.indexing.wasb.batch_size = 64
214         m.global_config.indexing.wasb.prefetch_batches = 4
215         fake = _FakeCloudClient()
216         _inject_fake(monkeypatch, fake)
217         m._execute_processing(
218             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
219         )
220         joined = " ".join(fake.calls[0])
221         assert "indexing.wasb.decode_backend=nvdec_resident" in joined
222         assert "indexing.wasb.batch_size=64" in joined
223         assert "indexing.wasb.prefetch_batches=4" in joined
224
225     def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
226         """decode_backend defaults to cpu (always forwarded ? harmless, keeps worker ==
227         control-plane),
228         but batch_size/prefetch_batches default to None and must NOT be forwarded (None =
229         the worker's
230         parity default; forwarding an empty value would reach argparse as a bad token)."""
231         _db, _bucket, monkeypatch = cloud_env
232         # defaults: decode_backend="cpu", batch_size=None, prefetch_batches=None
233         fake = _FakeCloudClient()
234         _inject_fake(monkeypatch, fake)
235         m._execute_processing(
236             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
237         )
238         joined = " ".join(fake.calls[0])
239         assert "indexing.wasb.decode_backend=cpu" in joined
240         assert "indexing.wasb.batch_size" not in joined
241         assert "indexing.wasb.prefetch_batches" not in joined
242
243     def test_cloud_failure_marks_failed_no_rows(cloud_env):
244         db, _bucket, monkeypatch = cloud_env
245         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
246         confirm).
247         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
248         fake = _FakeCloudClient(returncode=1)
249         _inject_fake(monkeypatch, fake)
250
251         out = m._execute_processing(
252             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
253         )
254         assert out["status"] == "failed"
255         assert "returncode 1" in out["message"]
256         assert out["play_segments"] == []
257         assert db.query_play_segments(out["video_id"], {}) == []
258         # It DID dispatch + run remotely (then failed rc=1), so the path is honestly

```

```

cloud_run + dispatched.
254     assert (
255         out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
256     )
257
258
259     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
260         _db, _bucket, monkeypatch = cloud_env
261         fake = _FakeCloudClient()
262         _inject_fake(monkeypatch, fake)
263         local_path = str(tmp_path / "local_only.mp4")
264
265         out = m._execute_processing(
266             m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
267         )
268         assert out["status"] == "failed"
269         assert "cloud input" in out["message"]
270         assert fake.calls == [] # never dispatched a local-only path
271
272
273     def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
274         """A browser upload lands on THIS box's local disk; under cloud-serving with a CLOUD
input_root it
275         is STAGED to the bucket and the L4 job dispatches from the staged gs:// URI (not
hard-failed) ? #461.
276         The DB video record is repointed at the staged source so a later /api/compile can
re-fetch it."""
277         db, _bucket, _mp = cloud_env
278         m.global_config.storage.input_backend = "gcs"
279         m.global_config.storage.input_root = "gs://in-bucket/videos"
280         puts: list = []
281
282         def _fake_put(src, uri, **kw):
283             puts.append((str(src), uri, kw.get("config")))
284             return uri
285
286         monkeypatch.setattr(m.storage, "put", _fake_put)
287         fake = _FakeCloudClient()
288         _inject_fake(monkeypatch, fake)
289         local_path = str(tmp_path / "upload.mp4")
290
291         out = m._execute_processing(
292             m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
293         )
294
295         assert out["status"] == "success"
296         # 1) the local upload was staged UP to the input bucket, namespaced by video_id
297         assert len(puts) == 1
298         src, dest, put_cfg = puts[0]
299         assert src == local_path
300         assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
301         # the active storage settings are threaded into put() (so a non-default S3 endpoint
/ GCS project /
302         # etc. survives) ? not a fresh default client. The model_dump carries
input_root/input_backend.
303         assert put_cfg is not None and put_cfg.get("input_root") == "gs://in-bucket/videos"
304         assert put_cfg.get("input_backend") == "gcs"
305         # 2) the L4 job dispatched from the STAGED gs:// URI, not the unreadable local path
306         assert len(fake.calls) == 1
307         argv = fake.calls[0]
308         assert argv[argv.index("--video-uri") + 1] == dest
309         # 3) the DB video record now points at the staged source (so compile re-fetches it)
310         row = db.get_video(out["video_id"])
311         assert row is not None and row["filepath"] == dest
312

```

```

313
314     def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
315         cloud_env, tmp_path, monkeypatch
316     ):
317         """input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid
cloud config ?
318         the storage layer classifies an explicit root by its scheme (resolve_input_uri joins
under
319         input_root before ever consulting input_backend), so the staging gate must too (PR
#467 review).
320         Regression: the gate previously required input_backend != 'local' and hard-failed
this shape."""
321         _db, _bucket, _mp = cloud_env
322         m.global_config.storage.input_root = "gs://in-bucket/videos"
323         # input_backend deliberately NOT set ? stays at the model default 'local'
324         assert m.global_config.storage.input_backend == "local"
325         puts: list = []
326
327         def _fake_put(src, uri, **kw):
328             puts.append((str(src), uri))
329             return uri
330
331         monkeypatch.setattr(m.storage, "put", _fake_put)
332         fake = _FakeCloudClient()
333         _inject_fake(monkeypatch, fake)
334         local_path = str(tmp_path / "upload.mp4")
335
336         out = m._execute_processing(
337             m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
338         )
339
340         assert out["status"] == "success" # NOT the 'needs a cloud input' pre-dispatch
failure
341         assert len(puts) == 1
342         src, dest = puts[0]
343         assert src == local_path
344         assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
345         # dispatch continued from the STAGED gs:// URI
346         assert len(fake.calls) == 1
347         argv = fake.calls[0]
348         assert argv[argv.index("--video-uri") + 1] == dest
349
350
351     def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
352         """No cloud input_root configured ? we genuinely can't stage a local upload ? hard-
fail (no
353         dispatch), rather than dispatching a path the Cloud Run job can't read."""
354         _db, _bucket, _mp = cloud_env # fixture leaves input_root unset
(input_backend=local)
355         fake = _FakeCloudClient()
356         _inject_fake(monkeypatch, fake)
357         called = []
358         monkeypatch.setattr(m.storage, "put", lambda *a, **k: called.append(a))
359
360         out = m._execute_processing(
361             m.ProcessRequest(video_path=str(tmp_path / "up.mp4"), segmenter="wasb_hybrid")
362         )
363         assert out["status"] == "failed"
364         assert "cloud input" in out["message"]
365         assert fake.calls == [] and called == [] # neither staged nor dispatched
366
367
368     def test_default_off_runs_in_process(tmp_path, monkeypatch):
369         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter

```

```

runs
370 byte-identically (the gate is inert). No Cloud Run dispatch."""
371 db = Database(str(tmp_path / "t.db")) # noqa: F841
372 monkeypatch.setattr(m, "db_instance", db)
373 monkeypatch.setattr(m, "db_instance", db)
374 cfg = AppConfig() # compute_target defaults to "" # noqa: F841
375 monkeypatch.setattr(m, "global_config", cfg)
376 monkeypatch.setattr(m, "global_config", cfg)
377 monkeypatch.setattr(m, "global_config", cfg)
378 monkeypatch.setattr(
379     m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
380 )
381 video = tmp_path / "v.mp4"
382 video.write_bytes(b"hashable-bytes")
383
384 class _InProcSeg:
385     def __init__(self, db_, cfg_):
386         self.db = db_
387
388     def process_video(self, vid, path, *a, **k):
389         sid = self.db.add_play_segment(vid, 0.0, 5.0)
390         return [
391             {
392                 "id": sid,
393                 "start_time": 0.0,
394                 "end_time": 5.0,
395                 "duration": 5.0,
396                 "metadata": {},
397             }
398         ]
399
400 monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
401 out = m._execute_processing(
402     m.ProcessRequest(video_path=str(video), segmenter="motion")
403 )
404 assert out["status"] == "success"
405 assert len(out["play_segments"]) == 1
406 assert "cloud_run" not in out["message"]
407
408
409 def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
410     """A rc=0 marker but a MISSING intervals.csv (a partial push) must fail cleanly +
leak no rows
411     (destroy-AFTER-confirm), not 500 or hang."""
412     db, _bucket, monkeypatch = cloud_env
413
414     class _NoCsvClient(_FakeCloudClient):
415         def run_job(self, job_name, argv, *, region, project=None):
416             self.calls.append(list(argv))
417             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
418             done_uri = argv[argv.index("--done-uri") + 1]
419             outputs = os.path.join(out_uri, "outputs", self.video_id)
420             os.makedirs(outputs, exist_ok=True)
421             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
422                 json.dump({"video_id": self.video_id, "status": "success"}, f)
423             # deliberately DO NOT write intervals.csv
424             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
425             with open(done_uri, "w", encoding="utf-8") as f:
426                 json.dump({"video_id": self.video_id, "returncode": 0}, f)
427             return "exec"
428
429     _inject_fake(monkeypatch, _NoCsvClient())
430     out = m._execute_processing(
431         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
432     )

```

```

433         assert out["status"] == "failed"
434         assert "could not read the result" in out["message"]
435         assert db.query_play_segments(out["video_id"], {}) == []
436
437
438     def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
439         """If the worker re-hashes different bytes (result.video_id != local id), ingest
still proceeds
440         under the LOCAL id but logs a WARNING so the orphaned-report condition is
diagnosable."""
441         db, _bucket, monkeypatch = cloud_env
442         _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
443         with caplog.at_level("WARNING", logger="backend.main"):
444             out = m._execute_processing(
445                 m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
446             )
447         assert out["status"] == "success"
448         assert len(db.query_play_segments(out["video_id"], {})) == 2
449         assert any(
450             "DIVERGED" in r.getMessage() and "!= local" in r.getMessage()
451             for r in caplog.records
452         )
453
454
455     class _InProcSeg:
456         """Minimal in-process segmenter (writes one play row) ? proves the local path
actually ran."""
457
458         def __init__(self, db_, cfg_):
459             self.db = db_
460
461         def process_video(self, vid, path, *a, **k):
462             sid = self.db.add_play_segment(vid, 0.0, 5.0)
463             return [
464                 {
465                     "id": sid,
466                     "start_time": 0.0,
467                     "end_time": 5.0,
468                     "duration": 5.0,
469                     "metadata": {},
470                 }
471             ]
472
473
474     # --- item 3: SPA compute-picker (per-request compute override + cloud_available
capability) ---
475
476
477     def test_per_request_local_forces_in_process(cloud_env):
478         """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
479         forces the in-process segmenter and NEVER dispatches to Cloud Run."""
480         db, _bucket, monkeypatch = cloud_env
481         fake = _FakeCloudClient()
482         _inject_fake(monkeypatch, fake)
483         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
484
485         out = m._execute_processing(
486             m.global_config,
487             m.db_instance,
488             m.ProcessRequest(
489                 video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
490             ),
491         )
492         assert out["status"] == "success"

```

```

493     assert len(out["play_segments"]) == 1
494     assert "cloud_run" not in out["message"]
495     assert (
496         fake.calls == []
497     ) # explicit "run on my machine" ? no dispatch despite a cloud server
498
499
500     def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
501         """Server default is LOCAL (compute_target=''), but a per-request compute='cloud'
502         forces a Cloud
503         Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT +
504         a bucket)."""
505         db = Database(str(tmp_path / "t.db")) # noqa: F841
506         monkeypatch.setattr(m, "db_instance", db)
507         bucket = tmp_path / "bucket"
508         bucket.mkdir()
509         cfg = AppConfig.model_validate(
510             {
511                 # noqa: F841
512                 "deployment": {"compute_target": ""}, # server default = in-process
513                 "storage": {"backend": "local", "output_root": str(bucket)},
514             }
515         )
516         monkeypatch.setattr(m, "global_config", cfg)
517         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
518         fake_local = tmp_path / "fetched.mp4"
519         fake_local.write_bytes(b"FAKEVIDEOBYTES")
520         monkeypatch.setattr(
521             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
522         )
523         monkeypatch.setattr(
524             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
525         )
526         monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
527         monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
528         fake = _FakeCloudClient()
529         _inject_fake(monkeypatch, fake)
530
531         out = m._execute_processing(
532             m.global_config,
533             m.db_instance,
534             m.ProcessRequest(
535                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
536             ),
537         )
538         assert out["status"] == "success"
539         assert len(out["play_segments"]) == 2
540         assert (
541             len(fake.calls) == 1
542         ) # per-request override dispatched despite the LOCAL server default
543
544
545     def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
546         """compute='cloud' on a server NOT wired for it fails clearly (no silent local
547         fallback, no
548         dispatch) so the picker can surface 'cloud isn't available here'."""
549         db = Database(str(tmp_path / "t.db")) # noqa: F841
550         monkeypatch.setattr(m, "db_instance", db)
551         cfg = AppConfig.model_validate(
552             {"deployment": {"compute_target": ""}}
553         ) # no bucket, no env # noqa: F841
554         monkeypatch.setattr(m, "global_config", cfg)
555         monkeypatch.setattr(
556             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
557         )
558         monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)

```

```

555 monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
556 video = tmp_path / "v.mp4"
557 video.write_bytes(b"hashable-bytes")
558
559 out = m._execute_processing(
560     m.global_config,
561     m.db_instance,
562     m.ProcessRequest(
563         video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
564     ),
565 )
566 assert out["status"] == "failed"
567 assert "isn't configured" in out["message"]
568
569
570 def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
571     """An unrecognised compute value is ignored (logged) and the server default applies
? here a
572     cloud_run server still dispatches to the cloud."""
573     db, _bucket, monkeypatch = cloud_env
574     fake = _FakeCloudClient()
575     _inject_fake(monkeypatch, fake)
576     with caplog.at_level("WARNING", logger="backend.main"):
577         out = m._execute_processing(
578             m.global_config,
579             m.db_instance,
580             m.ProcessRequest(
581                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
582             ),
583         )
584     assert out["status"] == "success"
585     assert len(fake.calls) == 1 # fell back to the configured cloud_run default
586     assert any("unknown compute" in r.getMessage() for r in caplog.records)
587
588
589 def test_capabilities_reports_cloud_available(cloud_env):
590     """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA
shows the
591     cloud toggle only then)."""
592     _db, _bucket, _monkeypatch = cloud_env
593     app = m.create_app(m.global_config, m.db_instance)
594     client = TestClient(app)
595     resp = client.get("/api/capabilities")
596     caps = resp.json()
597     assert caps["deployment"]["cloud_available"] is True
598     assert caps["deployment"]["compute_target"] == "cloud_run"
599
600
601 def test_capabilities_cloud_unavailable_by_default(monkeypatch, tmp_path):
602     """A default (local) server with no Cloud Run env advertises
cloud_available=False."""
603     cfg = AppConfig()
604     db = Database(str(tmp_path / "t.db"))
605     app = m.create_app(cfg, db)
606     monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
607     monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
608     client = TestClient(app)
609     resp = client.get("/api/capabilities")
610     caps = resp.json()
611     assert caps["deployment"]["cloud_available"] is False
612
613
614 def test_full_api_process_to_query_via_cloud(cloud_env):
615     """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches

```

```

616 to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
617 db, _bucket, monkeypatch = cloud_env
618 fake = _FakeCloudClient(video_id="vidCloud2")
619 _inject_fake(monkeypatch, fake)
620 app = m.create_app(m.global_config, db)
621 client = TestClient(app)
622
623 r = client.post(
624     "/api/process",
625     json={"video_path": "gs://b/clip.mp4", "segmenter": "wasb_hybrid"},
626 )
627 assert r.status_code == 202
628 job_id = r.json()["job_id"]
629
630 job = client.get(f"/api/jobs/{job_id}").json()
631 assert job["status"] == "done"
632 video_id = job["video_id"]
633 assert video_id
634
635 q = client.post("/api/query", json={"video_id": video_id})
636 assert q.status_code == 200
637 segs = q.json()["play_segments"]
638 assert len(segs) == 2
639 assert all(_meta(s)["source"] == "cloud_run" for s in segs)
640
641
642 def test_local_override_survives_api_boundary(cloud_env):
643     """REGRESSION: the picker choice must survive the async submit?worker?reconstruct
644     boundary. On a
645     cloud_run server, POST /api/process with compute='local' must persist (jobs.compute)
646     + reconstruct
647     (_job_to_request) the choice and run IN-PROCESS ? not dispatch to Cloud Run. (An
648     earlier cut
649     dropped compute in create_job/_job_to_request, silently nullifying the picker
650     through /api/process.)"""
651     db, _bucket, monkeypatch = cloud_env
652     fake = _FakeCloudClient()
653     _inject_fake(monkeypatch, fake)
654     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
655     app = m.create_app(m.global_config, db)
656     client = TestClient(app)
657
658     r = client.post(
659         "/api/process",
660         json={
661             "video_path": "gs://b/clip.mp4",
662             "segmenter": "motion",
663             "compute": "local",
664         },
665     )
666     assert r.status_code == 202
667     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
668     assert job["status"] == "done"
669     assert (
670         fake.calls == []
671     ) # 'local' honoured across the boundary ? never dispatched to Cloud Run
672     segs = db.query_play_segments(job["video_id"], {})
673     assert len(segs) == 1
674     assert all(
675         _meta(s).get("source") != "cloud_run" for s in segs
676     ) # in-process rows, not cloud
677
678
679 def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
680     """REGRESSION (mirror): on a LOCAL-default server wired for cloud (env + bucket),

```

```

POST /api/process
677     with compute='cloud' must survive the boundary and dispatch to Cloud Run.""
678     db = Database(str(tmp_path / "t.db")) # noqa: F841
679     monkeypatch.setattr(m, "db_instance", db)
680     bucket = tmp_path / "bucket"
681     bucket.mkdir()
682     cfg = AppConfig.model_validate(
683         { # noqa: F841
684             "deployment": {"compute_target": ""}, # server default = in-process
685             "storage": {"backend": "local", "output_root": str(bucket)},
686         }
687     )
688     monkeypatch.setattr(m, "global_config", cfg)
689     monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
690     fake_local = tmp_path / "fetched.mp4"
691     fake_local.write_bytes(b"FAKEVIDEOBYTES")
692     monkeypatch.setattr(
693         m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
694     )
695     monkeypatch.setattr(
696         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
697     )
698     monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
699     monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
700     fake = _FakeCloudClient(video_id="vidBoundary")
701     _inject_fake(monkeypatch, fake)
702     app = m.create_app(m.global_config, db)
703     client = TestClient(app)
704
705     r = client.post(
706         "/api/process",
707         json={
708             "video_path": "gs://b/clip.mp4",
709             "segmenter": "wasb_hybrid",
710             "compute": "cloud",
711         },
712     )
713     assert r.status_code == 202
714     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
715     assert job["status"] == "done"
716     assert (
717         len(fake.calls) == 1
718     ) # 'cloud' honoured across the boundary ? dispatched despite local default
719     segs = db.query_play_segments(job["video_id"], {})
720     assert len(segs) == 2
721     assert all(_meta(s)["source"] == "cloud_run" for s in segs)
722
723
724     # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
725     # -----
726     # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
727     # Run Job mounts
728     # them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
729     # there. It must fail
730     # FAST with an actionable "run in the cloud" message ? not fall through to the in-
731     # process segmenter
732     # that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
733     # 2026-07-07).
734
735     def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
736         """compute='local' + a weights-backed detector on a cloud_run server with NO local
737         weights ?
738         a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
739         found')."""

```

```

734     _db, _bucket, _mp = cloud_env
735     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
736     m.global_config.indexing.wasb.weights_path =
737     "/nonexistent/wasb_badminton_best.pth.tar"
738     fake = _FakeCloudClient()
739     _inject_fake(monkeypatch, fake)
740
741     out = m._execute_processing(
742         m.ProcessRequest(
743             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
744         )
745     )
746     assert out["status"] == "failed"
747     assert "can't run detection locally" in out["message"]
748     assert "Run in cloud" in out["message"]
749     assert "weights not found" not in out["message"] # NOT the cryptic deep failure
750     assert fake.calls == [] # never dispatched
751     # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
752     assert out["compute"]["requested"] == "local"
753     assert out["compute"]["dispatched"] is False
754     assert out["compute"]["path"] == "not_run"
755
756     def test_local_on_cloud_server_with_weights_present_runs_in_process(
757         cloud_env, tmp_path, monkeypatch
758     ):
759         """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
760         ACTUALLY has the
761         detector weights on disk, compute='local' still runs the in-process segmenter (no
762         dispatch)."""
763         _db, _bucket, _mp = cloud_env
764         weights = tmp_path / "wasb_badminton_best.pth.tar"
765         weights.write_bytes(b"WEIGHTS")
766         monkeypatch.delenv("WASB_WEIGHTS", raising=False)
767         m.global_config.indexing.wasb.weights_path = str(weights)
768         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
769         fake = _FakeCloudClient()
770         _inject_fake(monkeypatch, fake)
771
772         out = m._execute_processing(
773             m.ProcessRequest(
774                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
775             )
776         )
777         assert out["status"] == "success"
778         assert fake.calls == [] # in-process, not dispatched
779         assert out["compute"]["path"] == "in_process"
780
781     def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
782         cloud_env, monkeypatch
783     ):
784         """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
785         (motion) with
786         compute='local' on a cloud server is unaffected ? it runs in-process, never fails
787         fast."""
788         _db, _bucket, _mp = cloud_env
789         m.global_config.indexing.wasb.weights_path =
790         "/nonexistent/should_not_matter.pth.tar"
791         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
792         fake = _FakeCloudClient()
793         _inject_fake(monkeypatch, fake)
794
795         out = m._execute_processing(
796             m.ProcessRequest(

```

```

793         video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
794     )
795 )
796 assert out["status"] == "success"
797 assert fake.calls == []
798 assert out["compute"]["path"] == "in_process"
799
800
801 def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
802     tmp_path, monkeypatch
803 ):
804     """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
805     weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
806     'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
807     db = Database(str(tmp_path / "t.db")) # noqa: F841
808     monkeypatch.setattr(m, "db_instance", db)
809     cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa: F841
810     monkeypatch.setattr(m, "global_config", cfg)
811     monkeypatch.setattr(
812         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
813     )
814     monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
815     monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
816     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
817     cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
818     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
819     video = tmp_path / "v.mp4"
820     video.write_bytes(b"hashable-bytes")
821
822     out = m._execute_processing(
823         m.ProcessRequest(
824             video_path=str(video), segmenter="wasb_hybrid", compute="local"
825         )
826     )
827     # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
828     assert out["status"] == "success"
829     assert out["compute"]["path"] == "in_process"
830
831
832     # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
833
834
835     def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
836         """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run
execution name
837         (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the
intended path is
838         provable from the job result alone (no guessing from segment metadata)."""
839         _db, _bucket, monkeypatch = cloud_env
840         _inject_fake(monkeypatch, _FakeCloudClient())
841
842         out = m._execute_processing(
843             m.ProcessRequest(
844                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
845             )
846         )
847         prov = out["compute"]
848         assert prov["path"] == "cloud_run"
849         assert prov["requested"] == "cloud"
850         assert (
851             prov["execution"] == "exec-1"

```

```

852         ) # the fake client's execution name (real one in prod)
853         assert prov["outputs_uri"].endswith("/outputs/vidCloud/")
854         assert prov["job"] and prov["region"] # the dispatch coordinates are recorded
855
856
857     def test_local_override_stamps_in_process_provenance(cloud_env):
858         """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
859         requested=local ? so a 'ran local despite a cloud server' is auditable (no silent
ambiguity)."""
860         _db, _bucket, monkeypatch = cloud_env
861         _inject_fake(monkeypatch, _FakeCloudClient())
862         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
863
864         out = m._execute_processing(
865             m.ProcessRequest(
866                 video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
867             )
868         )
869         assert out["status"] == "success"
870         assert out["compute"] == {"path": "in_process", "requested": "local"}
871
872
873     def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
874         """A default-OFF (no cloud) run stamps path=in_process, requested=None ? the byte-
identical local
875         path is now self-describing."""
876         db = Database(str(tmp_path / "t.db")) # noqa: F841
877         monkeypatch.setattr(m, "db_instance", db)
878         monkeypatch.setattr(m, "db_instance", db)
879         monkeypatch.setattr(m, "global_config", AppConfig())
880         m.app.state.config = AppConfig()
881         monkeypatch.setattr(
882             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
883         )
884         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
885         video = tmp_path / "v.mp4"
886         video.write_bytes(b"hashable-bytes")
887
888         out = m._execute_processing(
889             m.ProcessRequest(video_path=str(video), segmenter="motion")
890         )
891         assert out["compute"] == {"path": "in_process", "requested": None}
892
893
894     def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
895         """compute='cloud' on a server that can't dispatch fails clearly AND records it:
target cloud_run,
896         requested=cloud, dispatched=False ? proving the cloud job never ran (no silent local
fallback)."""
897         db = Database(str(tmp_path / "t.db")) # noqa: F841
898         monkeypatch.setattr(m, "db_instance", db)
899         monkeypatch.setattr(m, "db_instance", db)
900         monkeypatch.setattr(
901             m,
902             "global_config",
903             AppConfig.model_validate({"deployment": {"compute_target": ""}}),
904         )
905         m.app.state.config = AppConfig.model_validate(
906             {"deployment": {"compute_target": ""}}
907         )
908         monkeypatch.setattr(
909             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
910         )
911         monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)

```

```

912 monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
913 video = tmp_path / "v.mp4"
914 video.write_bytes(b"hashable-bytes")
915
916 out = m._execute_processing(
917     m.ProcessRequest(
918         video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
919     )
920 )
921 assert out["status"] == "failed"
922 # Honest: it ran NOWHERE (pre-dispatch reject) ? path=not_run, but the intent is
recorded.
923 assert out["compute"] == {
924     "path": "not_run",
925     "requested": "cloud",
926     "target": "cloud_run",
927     "dispatched": False,
928 }
929
930
931 def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
932     """A validation failure never reaches a segmenter ? path=not_run (nothing
executed)."""
933     db = Database(str(tmp_path / "t.db")) # noqa: F841
934     monkeypatch.setattr(m, "db_instance", db)
935     monkeypatch.setattr(m, "global_config", AppConfig())
936     m.app.state.config = AppConfig()
937     fail = [
938         ValidationResult(validator_name="x", passed=False, message="bad", details={})
939     ]
940     monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (fail, {}))
941     video = tmp_path / "v.mp4"
942     video.write_bytes(b"hashable-bytes")
943
944     out = m._execute_processing(m.ProcessRequest(video_path=str(video)))
945     assert out["status"] == "failed"
946     assert out["compute"]["path"] == "not_run"
947
948
949 def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
950     """End-to-end: POST /api/process (compute=cloud) ? the job result the SPA polls
carries the
951     cloud_run provenance + execution, so the UI/operator can PROVE the cloud path was
taken."""
952     _db, _bucket, monkeypatch = cloud_env
953     _inject_fake(monkeypatch, _FakeCloudClient(video_id="vidProv"))
954     app = m.create_app(m.global_config, _db)
955     client = TestClient(app)
956
957     r = client.post(
958         "/api/process",
959         json={
960             "video_path": "gs://b/clip.mp4",
961             "segmenter": "wasb_hybrid",
962             "compute": "cloud",
963         },
964     )
965     assert r.status_code == 202
966     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
967     assert job["status"] == "done"
968     assert job["result"]["compute"]["path"] == "cloud_run"
969     assert job["result"]["compute"]["execution"] == "exec-1"
970
971
972 # --- #468: local compute lane ? CPU/GPU-heavy in-process work serializes, remote polls

```

```

don't -----
973
974
975     def test_in_process_detection_runs_under_the_local_lane(cloud_env):
976         """The in-process segmenter holds orchestration._LOCAL_COMPUTE_LANE while it runs,
so raising the
977         drain worker count (to overlap I/O-bound remote dispatch) never runs two local
detections at once.
978         A remote dispatch returns before this path, so remote polls never hold the lane
(they overlap)."""
979         import backend.orchestration as orch
980
981         _db, _bucket, monkeypatch = cloud_env
982         held = {}
983
984         class _LaneCheckSeg:
985             def __init__(self, db_, cfg_):
986                 self.db = db_
987
988             def process_video(self, vid, path, *a, **k):
989                 # the wrapping `with _LOCAL_COMPUTE_LANE` holds the size-1 lane ? a non-
blocking re-acquire
990                 # must FAIL while detection runs (proving concurrent local detections are
excluded).
991                 got = orch._LOCAL_COMPUTE_LANE.acquire(blocking=False)
992                 held["busy"] = not got
993                 if got:
994                     orch._LOCAL_COMPUTE_LANE.release()
995                     sid = self.db.add_play_segment(vid, 0.0, 5.0)
996                     return [
997                         {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}
998                     ]
999
1000         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _LaneCheckSeg)
1001         out = m._execute_processing(
1002             m.global_config,
1003             m.db_instance,
1004             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion",
compute="local"),
1005         )
1006         assert out["status"] == "success"
1007         assert held.get("busy") is True # detection ran while the local lane was held
1008
1009
1010     def test_wait_progress_message_is_honest_and_minute_granular():
1011         """#482: the CP knows ELAPSED time, not frames ? the heartbeat says so plainly,
shows
1012         movement from the first tick ("starting up" during queue/cold-start), then counts
minutes."""
1013         from backend.orchestration import _wait_progress_message
1014
1015         early = _wait_progress_message("wasb_hybrid", 12.0)
1016         assert "wasb_hybrid" in early and "starting up" in early
1017         later = _wait_progress_message("wasb_hybrid", 125.0)
1018         assert later.endswith("2 min elapsed")
1019

```

12. assistant (2026-07-07T15:08:51.181Z)

This is a rich test file. Now I need to verify the guardrail internals and the SPA tests. Let me read the orchestration source (for `\_failed`, `\_cloud\_available`, `\_maybe\_dispatch\_remote`, and provenance) and the SPA test file.

13. user (2026-07-07T15:08:54.971Z)

```

190: def _cloud_available(cfg) -> bool:
243:     video_id: str, validation_results, segments, requested_compute
257:         "compute": {"path": "cached", "requested": requested_compute},
321:     db, cfg, video_id: str, video_path: str, requested_compute=None
345:         requested_compute,
388:         return _cached_process_payload(video_id, [], segments, requested_compute)
423: def _maybe_dispatch_remote(
445:     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
450:         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker rc /
ingest
451:         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
452:         # intended backend, `requested` the picker choice, `dispatched` whether the remote
job ran.
461:             "path": "cloud_run" if ran else "not_run",
462:             "requested": req.compute,
464:             "dispatched": ran,
502:             "cloud-serving was requested but this server isn't configured for it "
674:             "requested": req.compute,
681:             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s video_id=%s",
701: def _execute_processing(
900:     # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
901:     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is then
visible.
906:         "path": compute_actual or "not_run",
907:         "requested": getattr(req, "compute", None),
912:         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
916:         prov.get("dispatched"),
934:         segmenter_requested=req.segmenter,
981:     row could change between submit and run). Verifies the video exists, the requested
segments match

```

14. user (2026-07-07T15:08:55.993Z)

```

.claude\worktrees\zealous-pike-7d0a85\web\src\components\IngestPanel.test.tsx
web\src\components\IngestPanel.test.tsx

```

15. assistant (2026-07-07T15:08:59.952Z)

Let me read the key orchestration sections and the SPA test file.

16. user (2026-07-07T15:09:02.055Z)

```

185         or ""
186     )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190     def _cloud_available(cfg) -> bool:
191         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
192
193         True when the control plane is wired for Cloud Run: either the configured default
target is
194         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196         are present so a forced per-request override can actually dispatch + collect a
result.
197
198         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""

```

```

199     deployment = getattr(cfg, "deployment", None)
200     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201         return True
202     has_job = bool(
203         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204     )
205     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206     return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
212         comes
213         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
214         "starting"
215         (queue/cold-start), so the very first tick already shows movement."""
216         minutes = int(elapsed_sec) // 60
217         if minutes < 1:
218             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
219         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
220
221     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
222         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
223         gs://
224         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
225         keep
226         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
227         no
228         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
229         if getattr(input_ref, "backend", "") != "gcs":
230             return None
231         sc = _storage_settings(cfg)
232         if not sc:
233             return None
234         try:
235             st = storage.get_storage("gcs", config=sc).stat(input_ref)
236             return getattr(st, "md5", None) or None
237         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
238             logger.info(
239                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
240                 getattr(input_ref, "uri", input_ref),
241                 exc_info=True,
242             )
243             return None
244
245     def _cached_process_payload(
246         video_id: str, validation_results, segments, requested_compute
247     ) -> Dict[str, Any]:
248         """The /api/process result for a video whose detection already exists ? ONE shape
249         for
250         both producers (the drain's DB-cache short-circuit and the submit-time cache,
251         #484)."""
252         return {
253             "video_id": video_id,
254             "status": "success",
255             "message": (
256                 f"Reused {len(segments)} cached play segments. "
257                 "Pass force=true to reprocess this video."
258             ),
259             "results": validation_results or [],
260             "play_segments": segments,
261             "cached": True,

```

```

257         "compute": {"path": "cached", "requested": requested_compute},
258     }
259
260
261     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
264         ref = storage.parse_uri(uri)

```

17. user (2026-07-07T15:09:03.102Z)

```

423     def _maybe_dispatch_remote(
424         cfg,
425         db,
426         req: "ProcessRequest",
427         video_id: str,
428         seg_name: str,
429         val_results,
430         play_wm: int,
431         cand_wm: int,
432         notify,
433     ) -> Optional[Tuple[Dict[str, Any], bool]]:
434         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
435         == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
436         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
437
438         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
439         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
440         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
441         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
442         the in-process path stays byte-identical)."""
443         from backend.compute import ComputeJobSpec, get_compute_backend
444
445         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
446             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
447             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
448             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
449             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
450             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
451             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
452             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
453             return (
454                 {
455                     "video_id": video_id,
456                     "status": "failed",
457                     "message": msg,
458                     "results": [r.model_dump() for r in val_results],
459                     "play_segments": [],
460                     "compute": {
461                         "path": "cloud_run" if ran else "not_run",
462                         "requested": req.compute,
463                         "target": "cloud_run",
464                         "dispatched": ran,

```

```

465         },
466     },
467     ran,
468 )
469
470     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473     choice = (req.compute or "").strip().lower()
474     if choice and choice not in ("local", "cloud"):
475         logger.warning(
476             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477             req.compute,
478         )
479     choice = ""
480     if choice == "local":
481         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487         # below exactly as before (the weights-present check is the escape hatch).
488         if (
489             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490             and _cloud_available(cfg)
491             and not _local_detector_weights_present(cfg, seg_name)
492         ):
493             return _failed(
494                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496                 False,
497             )
498             return None # explicit "run on my machine" ? in-process regardless of the
server's target
499         if choice == "cloud":
500             if not _cloud_available(cfg):
501                 return _failed(
502                     "cloud-serving was requested but this server isn't configured for it "
503                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                     False,
505                 )
506             compute = get_compute_backend(cfg, target_override="cloud_run")
507         else:
508             compute = get_compute_backend(
509                 cfg
510             ) # server default (default-OFF unless cloud_run)
511         if compute is None:
512             return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514     storage_cfg = getattr(cfg, "storage", None)
515

```

```

516         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517         try:
518             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519         except ValueError as e:
520             return _failed(
521                 f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522             )
523         if input_ref.backend == "local":
524             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525             # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526             # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527             # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528             # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529             # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530             # default, while a bare/local-looking root can't stage no matter what
input_backend says
531             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532             # gives us nothing to build the staged URI under).

```

18. assistant (2026-07-07T15:09:16.937Z)

Let me read `\_execute\_processing` and check how `backend.main` exposes it (the guardrail tests call it single-arg, others 3-arg).

19. user (2026-07-07T15:09:20.130Z)

```

701     def _execute_processing(
702         config=None, db=None, req=None, progress=None
703     ) -> Dict[str, Any]:
704         """The full hash ? validate ? segment ? report pipeline for one video.
705
706         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
707         module-level ``global_config``/``db_instance`` (backward-compatible with existing
708         tests that pass only ``req``). Route handlers always pass explicit params; older
709         test code that calls ``_execute_processing(req)`` still works.
710
711         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
712         Raises on unexpected internal errors ? the caller records those as a job failure
713         """
714         import backend.main as _main # call-time seam: main's globals/model stay patchable
715
716         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
717         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
718         if config is not None and not isinstance(config, AppConfig):
719             # Old signature: _execute_processing(req, progress=None)
720             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
721                 req = db
722                 db = None
723             elif req is None and isinstance(config, _main.ProcessRequest):
724                 req = config
725                 config = None
726         # Resolve config/db from module globals when not passed explicitly
727         if config is None:
728             config = _main.global_config
729         if db is None:

```

```

730         db = _main.db_instance
731
732     notify = progress or (lambda stage: None)
733
734     notify("hashing")
735     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
736     # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
737     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
738     # consumers (hash / validators / segmenter) use the resolved local path.
739     local_video = ""
740     try:
741         local_video = storage.acquire_local_input(
742             req.video_path, getattr(config, "storage", None)
743         )
744         video_id = _main.compute_video_id(local_video)
745     except Exception:
746         storage.cleanup_acquired_local_input(
747             req.video_path, local_video, getattr(config, "storage", None)
748         )
749         raise
750     try:
751         existing_video = db.get_video(video_id)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     was_reingest = existing_video is not None
758
759     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
760     default_seg = getattr(
761         getattr(config, "indexing", None), "default_segmenter", "motion"
762     )
763     seg_name = req.segmenter or default_seg
764     segmenter_actual = None
765     segmentation_ran = False
766     val_results: List[Any] = []
767     val_context: Dict[str, Any] = {}
768     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
769     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
770     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
771     compute_actual: Optional[str] = None
772     response: Optional[Dict[str, Any]] = None
773     try:
774         if (
775             existing_video
776             and existing_video.get("status") == "processed"
777             and not req.force
778         ):
779             cached_segments = db.query_play_segments(video_id, {})
780             if cached_segments:

```

20. user (2026-07-07T15:09:23.429Z)

```

=== git status ===
perf/validation-shared-decode-pass
=== _execute_processing in main.py ===
231:# backend.main ? `import backend.main as m; m._execute_processing(...)` , the job worker's
238:     _execute_processing,

```

```
=== def _execute_processing signature in main.py ===
```

21. assistant (2026-07-07T15:09:36.977Z)

Now let me read the SPA test file to assess the local-appliance (over-defaulting) coverage.

22. user (2026-07-07T15:09:37.400Z)

```
1      // @vitest-environment jsdom
2      //
3      // The ingest screen end to end: type a URI ? submit ? (mocked engine) ? onLoaded gets
the video_id;
4      // and the two failure surfaces a non-technical user actually hits ? the engine's own
400 detail,
5      // and a friendly "is the engine running?" when it's unreachable. Mounts the real i18n
catalog.
6      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7      import { render, screen, fireEvent, cleanup, act, waitFor } from "@testing-
library/react";
8      import i18n from "../i18n/index";
9      import { IngestPanel } from "../IngestPanel";
10     import type { JobStatus } from "../api/types";
11
12     type FetchFn = typeof fetch;
13
14     function httpOk(status: number, json: unknown) {
15         return { ok: status >= 200 && status < 300, status, json: async () => json, text:
async () => JSON.stringify(json) };
16     }
17
18     function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
19         let i = 0;
20         return vi.fn(async (url: string | URL, init?: RequestInit) => {
21             const u = String(url);
22             // The panel probes /api/capabilities on mount (compute-picker) ? answer it benignly
so the
23             // ingest-flow assertions don't see a stray "unexpected fetch". No cloud ? no
picker.
24             if (u === "/api/capabilities") return httpOk(200, {});
25             if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
26             if (u.startsWith("/api/jobs/")) {
27                 const body = jobs[Math.min(i, jobs.length - 1)];
28                 i += 1;
29                 return httpOk(200, body);
30             }
31             throw new Error(`unexpected fetch ${u}`);
32         }) as unknown as FetchFn;
33     }
34
35     /** Like fetchSeq but lets a test set the capabilities payload + capture the
/api/process body ? used
36     * to drive and assert the compute-picker. */
37     function fetchAll(opts: {
38         caps?: unknown;
39         process?: unknown;
40         jobs?: JobStatus[];
41         onProcessBody?: (body: Record<string, unknown>) => void;
42     }): FetchFn {
43         let i = 0;
44         const jobs = opts.jobs ?? [];
45         return vi.fn(async (url: string | URL, init?: RequestInit) => {
46             const u = String(url);
47             if (u === "/api/capabilities") return httpOk(200, opts.caps ?? {});
48             if (u === "/api/process" && (init?.method ?? "GET") === "POST") {
49                 opts.onProcessBody?.(JSON.parse(String(init?.body ?? "{}"))) as Record<string,
```

```

unknown>);
50     return opts.process ?? httpOk(202, { job_id: "j", status: "queued", poll:
"/api/jobs/j" });
51     }
52     if (u.startsWith("/api/jobs/")) {
53         const body = jobs[Math.min(i, jobs.length - 1)];
54         i += 1;
55         return httpOk(200, body);
56     }
57     throw new Error(`unexpected fetch ${u}`);
58 } as unknown as FetchFn;
59 }
60
61 beforeEach(async () => {
62     window.localStorage.clear();
63     window.history.replaceState(null, "", "/");
64     await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
65 });
66
67 afterEach(() => {
68     cleanup();
69     vi.restoreAllMocks();
70     vi.unstubAllGlobals();
71     vi.useRealTimers();
72 });
73
74 const typeUri = (value: string) =>
75     fireEvent.change(screen.getByLabelText(/video location/i), { target: { value } });
76 const clickProcess = () => fireEvent.click(screen.getByRole("button", { name: /process
video/i }));
77 const clickStartUpload = () => fireEvent.click(screen.getByRole("button", { name: /start
upload and find rallies/i }));
78
79 describe("IngestPanel", () => {
80     it("submits a URI, polls, and hands the video_id to onLoaded", async () => {
81         vi.useFakeTimers();
82         vi.stubGlobal(
83             "fetch",
84             fetchSeq(httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" }), [
85                 { job_id: "j", status: "running", progress: "segmenting" },
86                 { job_id: "j", status: "done", video_id: "vidZ", result: { status: "success",
video_id: "vidZ" } } ],
87             ),
88         );
89         const onLoaded = vi.fn();
90         render(<IngestPanel onLoaded={onLoaded} />);
91         typeUri("gs://bucket/match.mp4");
92         clickProcess();
93
94         await act(async () => {
95             await vi.advanceTimersByTimeAsync(0); // flush submit ? start polling
96         });
97         await act(async () => {
98             await vi.advanceTimersByTimeAsync(1000); // running tick
99         });
100         await act(async () => {
101             await vi.advanceTimersByTimeAsync(1500); // done tick
102         });
103         expect(onLoaded).toHaveBeenCalledWith("vidZ", "match.mp4");
104     });
105
106     it("shows the engine's own 400 detail (e.g. an unsupported scheme)", async () => {
107         vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" })), []));

```

```

108     render(<IngestPanel onLoad={vi.fn()} />);
109     typeUri("ftp://nope");
110     clickProcess();
111     expect(await screen.findByText(/unsupported storage URI scheme/i)).toBeTruthy();
112     fireEvent.click(screen.getByRole("button", { name: /try again/i }));
113     expect(screen.queryByTestId("ingest-error")).toBeNull();
114 });
115
116 it("shows a failed pipeline verdict as the engine's reason, not as no rallies", async
() => {
117     vi.useFakeTimers();
118     vi.stubGlobal(
119         "fetch",
120         fetchSeq(httpOk(202, { job_id: "jFail", status: "queued", poll: "/api/jobs/jFail"
121     })), [
122         { job_id: "jFail", status: "done", video_id: "vidFail", result: { status:
"failed", message: "Video failed validation: blur: blurry" } },
123     ],
124     );
125     render(<IngestPanel onLoad={vi.fn()} />);
126     typeUri("gs://bucket/blurry.mp4");
127     clickProcess();
128     await act(async () => {
129         await vi.advanceTimersByTimeAsync(0);
130     });
131     await act(async () => {
132         await vi.advanceTimersByTimeAsync(1000);
133     });
134     expect(screen.getByTestId("ingest-error").textContent).toMatch(/Video failed
validation: blur: blurry/i);
135     expect(screen.getByTestId("ingest-error").textContent).not.toMatch(/found no
rallies/i);
136 });
137
138 it("shows a friendly, localized message when the engine is unreachable", async () => {
139     vi.stubGlobal("fetch", vi.fn(async () => {
140         throw new Error("offline");
141     }) as unknown as FetchFn);
142     render(<IngestPanel onLoad={vi.fn()} />);
143     typeUri("gs://bucket/match.mp4");
144     clickProcess();
145     expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
146 });
147
148 it("rejects an empty URI with a hint and never submits to the engine", () => {
149     // The mount-time capabilities probe may fire; assert specifically that /api/process
never does.
150     const f = fetchAll({ caps: {} });
151     vi.stubGlobal("fetch", f);
152     render(<IngestPanel onLoad={vi.fn()} />);
153     clickProcess();
154     expect(screen.getByTestId("ingest-error")).toBeTruthy();
155     const submitted = (f as unknown as { mock: { calls: unknown[][] } }
156     ).mock.calls.some(
157         ([u] => String(u) === "/api/process",
158     );
159     expect(submitted).toBe(false);
160 });
161
162 // --- compute-picker (item 3)
163 -----
164
165 it("hides the compute picker when the server can't dispatch to the cloud", async () =>
{
166     vi.stubGlobal("fetch", fetchAll({ caps: { deployment: { cloud_available: false } }

```

```

    });
164     render(<IngestPanel onLoad={vi.fn()} />);
165     await act(async () => {}); // let the capabilities probe resolve
166     expect(screen.queryByTestId("compute-picker")).toBeNull();
167   });
168
169   it("defaults to local and sends compute=local when the cloud is available", async ()
=> {
170     const bodies: Record<string, unknown>[] = [];
171     vi.stubGlobal(
172       "fetch",
173       fetchAll({
174         caps: { deployment: { cloud_available: true } },
175         onProcessBody: (b) => bodies.push(b),
176         jobs: [{ job_id: "j", status: "done", video_id: "vidL", result: { status:
"success", video_id: "vidL" } }],
177       }),
178     );
179     render(<IngestPanel onLoad={vi.fn()} />);
180     await screen.findByTestId("compute-picker");
181     typeUri("gs://bucket/match.mp4");
182     clickProcess();
183     await act(async () => {});
184     expect(bodies).toHaveLength(1);
185     expect(bodies[0].compute).toBe("local");
186   });
187
188   it("defaults to CLOUD (never sends compute=local) on a hosted cloud_run control-
plane", async () => {
189     // A hosted control-plane bakes no detector weights, so LOCAL can't run there ? the
picker must
190     // default to CLOUD. The cost gate still applies (accept it, then the sole submit is
compute=cloud).
191     const bodies: Record<string, unknown>[] = [];
192     vi.stubGlobal(
193       "fetch",
194       fetchAll({
195         caps: { deployment: { cloud_available: true, compute_target: "cloud_run" } },
196         onProcessBody: (b) => bodies.push(b),
197         jobs: [{ job_id: "j", status: "done", video_id: "vidH", result: { status:
"success", video_id: "vidH" } }],
198       }),
199     );
200     render(<IngestPanel onLoad={vi.fn()} />);
201     await screen.findByTestId("compute-picker");
202     // Defaulted to cloud ? the cost note + ack are shown up front (LOCAL would hide
them).
203     expect(screen.getByTestId("compute-cost")).toBeTruthy();
204     typeUri("gs://bucket/match.mp4");
205     fireEvent.click(screen.getByRole("checkbox")); // accept the cost gate
206     clickProcess();
207     await act(async () => {});
208     expect(bodies).toHaveLength(1);
209     expect(bodies[0].compute).toBe("cloud"); // never "local" by default on the hosted
build
210   });
211
212   it("shows a pasted host-path basename while processing and clears it on cancel", async
() => {
213     vi.useFakeTimers();
214     vi.stubGlobal(
215       "fetch",
216       fetchAll({
217         caps: {},
218         jobs: [{ job_id: "j", status: "running", progress: "hashing" }],

```

```

219     }},
220   );
221   render(<IngestPanel onLoad={vi.fn()} />);
222
223   typeUri("C:\\videos\\Sunday Match.mov");
224   clickProcess();
225   await act(async () => {
226     await vi.advanceTimersByTimeAsync(0);
227   });
228
229   expect(screen.getByTestId("ingest-source").textContent).toContain("Sunday
Match.mov");
230   fireEvent.click(screen.getByRole("button", { name: /cancel/i }));
231   expect(screen.queryByTestId("ingest-source")).toBeNull();
232   });
233
234   it("shows a decoded URL basename while processing", async () => {
235     vi.useFakeTimers();
236     vi.stubGlobal(
237       "fetch",
238       fetchAll({
239         caps: {},
240         jobs: [{ job_id: "j", status: "running", progress: "hashing" }],
241       }),
242     );
243     render(<IngestPanel onLoad={vi.fn()} />);
244
245     typeUri("https://cdn.example/video/Sunday%20Final.mp4?download=1");
246     clickProcess();
247     await act(async () => {
248       await vi.advanceTimersByTimeAsync(0);
249     });
250
251     expect(screen.getByTestId("ingest-source").textContent).toContain("Sunday
Final.mp4");
252   });
253
254   // --- medium picker (P5, store-into-medium)
255   -----
256   const capsBucket = {
257     storage: {
258       default_medium: "local",
259       backends: [
260         { id: "local", kind: "local", label: "This computer", usable: true, free_gb: 100
},
261         { id: "gcs", kind: "gcs", label: "GCS", usable: true, free_gb: null },
262         { id: "s3", kind: "s3", label: "S3", usable: false, free_gb: null },
263         { id: "gdrive", kind: "gdrive", label: "Drive", usable: false, free_gb: null },
264       ],
265     },
266   };
267
268   // A fetch mock that answers the full store?process choreography (upload ? assets ?
process ? job).
269   function fetchStore(opts: {
270     caps?: unknown;
271     jobs?: JobStatus[];
272     onAssetBody?: (b: Record<string, unknown>) => void;
273     onProcessBody?: (b: Record<string, unknown>) => void;
274     holdPartIndex?: number;
275     holdPart?: Promise<void>;
276     onPartRequest?: (part: number) => void;
277     maxPartMb?: number;
278   }): FetchFn {

```

```

279     let ji = 0;
280     const jobs = opts.jobs ?? [];
281     return vi.fn(async (url: string | URL, init?: RequestInit) => {
282         const u = String(url);
283         const m = init?.method ?? "GET";
284         if (u === "/api/capabilities") return httpOk(200, { opts.caps ?? {} });
285         if (u === "/api/upload/init") return httpOk(200, { upload_id: "up1", max_part_mb:
opts.maxPartMb ?? 95, next_part: "/api/upload/up1/part/0" });
286         const partMatch = u.match(/^\/api\/upload\/up1\/part\/(\d+)\$/);
287         if (partMatch) {
288             const part = Number(partMatch[1]);
289             opts.onPartRequest?.(part);
290             if (opts.holdPartIndex === part) await opts.holdPart;
291             return httpOk(200, { upload_id: "up1", received_parts: part + 1, received_bytes:
(init?.body as Blob).size });
292         }
293         if (/^\/api\/upload\/up1\/complete$/.test(u)) return httpOk(200, { path:
"/srv/up/match.mp4", video_id: "mock", size_bytes: 3 });
294         if (u === "/api/assets" && m === "POST") {
295             opts.onAssetBody?.(JSON.parse(String(init?.body ?? "{}")) as Record<string,
unknown>);
296             return httpOk(201, { video_id: "vidS", medium: "gcs", stored_uri:
"gcs://b/videos/vidS.mp4", copied: true, status: "stored" });
297         }
298         if (u === "/api/process" && m === "POST") {
299             opts.onProcessBody?.(JSON.parse(String(init?.body ?? "{}")) as Record<string,
unknown>);
300             return httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" });
301         }
302         if (u.startsWith("/api/jobs/")) {
303             const b = jobs[Math.min(ji, jobs.length - 1)];
304             ji += 1;
305             return httpOk(200, b);
306         }
307         throw new Error(`unexpected fetch ${m} ${u}`);
308     }) as unknown as FetchFn;
309 }
310
311 it("hides the medium picker when only local is usable", async () => {
312     vi.stubGlobal("fetch", fetchAll({ caps: { storage: { backends: [{ id: "local", kind:
"local", label: "x", usable: true } ] } } }));
313     render(<IngestPanel onLoaded={vi.fn()} />);
314     await act(async () => {});
315     expect(screen.queryByTestId("medium-picker")).toBeNull();
316 });
317
318 it("shows the picker and gates the upload until a destination is given", async () => {
319     vi.stubGlobal("fetch", fetchStore({ caps: capsBucket }));
320     render(<IngestPanel onLoaded={vi.fn()} />);
321     await screen.findByTestId("medium-picker");
322     fireEvent.click(screen.getByTestId("medium-bucket"));
323     // Destination empty ? blocked: the required hint shows and the file input is
disabled.
324     expect(screen.getByTestId("medium-required")).toBeTruthy();
325     const fileInput = document.querySelector('input[type="file"]') as HTMLInputElement;
326     expect(fileInput.disabled).toBe(true);
327     // Provide a destination ? the gate lifts.
328     fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gcs://b/videos" } });
329     expect(screen.queryByTestId("medium-required")).toBeNull();
330     expect(fileInput.disabled).toBe(false);
331     expect(screen.getByText(/choose video for bucket/i)).toBeTruthy();
332     expect(screen.getByLabelText(/selected cloud bucket/i)).toBe(fileInput);
333     expect(screen.getByText("MP4 or MOV.")).toBeTruthy();
334     // The source-link form stays visible: a gs:// source is not the same thing as the

```

```

upload destination.
335     expect(screen.getByText(/or use a cloud link/i)).toBeTruthy();
336   });
337
338   it("lets a pasted gs:// source submit even when bucket storage is selected without a
destination", async () => {
339     const assetBodies: Record<string, unknown>[] = [];
340     const processBodies: Record<string, unknown>[] = [];
341     vi.stubGlobal(
342       "fetch",
343       fetchStore({
344         caps: capsBucket,
345         onAssetBody: (b) => assetBodies.push(b),
346         onProcessBody: (b) => processBodies.push(b),
347         jobs: [{ job_id: "j", status: "done", video_id: "vidLink", result: { status:
"success", video_id: "vidLink" } }],
348       }),
349     );
350     const onLoaded = vi.fn();
351     render(<IngestPanel onLoaded={onLoaded} />);
352     await screen.findByTestId("medium-picker");
353     fireEvent.click(screen.getByTestId("medium-bucket"));
354     expect(screen.getByTestId("medium-required")).toBeTruthy();
355
356     typeUri("gs://source-bucket/matches/final.mp4");
357     clickProcess();
358
359     await waitFor(() => expect(processBodies).toHaveLength(1));
360     expect(processBodies[0].video_path).toBe("gs://source-bucket/matches/final.mp4");
361     expect(assetBodies).toHaveLength(0);
362   });
363
364   it("replaces setup controls with upload status, size and ETA once a bucket upload
starts", async () => {
365     let releasePart: () => void = () => {};
366     const heldPart = new Promise<void>((resolve) => {
367       releasePart = resolve;
368     });
369     let secondPartStarted: () => void = () => {};
370     const secondPart = new Promise<void>((resolve) => {
371       secondPartStarted = resolve;
372     });
373     vi.stubGlobal(
374       "fetch",
375       fetchStore({
376         caps: { deployment: { cloud_available: true }, ...capsBucket },
377         maxPartMb: 1,
378         holdPartIndex: 1,
379         holdPart: heldPart,
380         onPartRequest: (part) => {
381           if (part === 1) secondPartStarted();
382         },
383         jobs: [{ job_id: "j", status: "done", video_id: "vidS", result: { status:
"success", video_id: "vidS" } }],
384       }),
385     );
386     render(<IngestPanel onLoaded={vi.fn()} />);
387     await screen.findByTestId("compute-picker");
388     fireEvent.click(screen.getByTestId("medium-bucket"));
389     fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gs://b/videos" } });
390     const file = new File([new Uint8Array(2 * 1024 * 1024)], "match.mp4", { type:
"video/mp4" });
391
392     await act(async () => {

```

```

393         fireEvent.change(document.querySelector('input[type="file"]') as HTMLInputElement,
{ target: { files: [file] } });
394     });
395
396     expect(screen.getByTestId("ingest-video-title").textContent).toContain("match.mp4");
397     expect(screen.getByTestId("selected-file").textContent).toContain("2 MB selected");
398     expect(screen.queryByTestId("upload-progress")).toBeNull();
399     expect(screen.getByTestId("compute-picker")).toBeTruthy();
400     expect(screen.getByTestId("medium-picker")).toBeTruthy();
401
402     await act(async () => {
403         clickStartUpload();
404         await secondPart;
405     });
406
407     expect(screen.getByTestId("upload-progress")).toBeTruthy();
408     expect(screen.queryByTestId("compute-picker")).toBeNull();
409     expect(screen.queryByTestId("medium-picker")).toBeNull();
410     expect(screen.queryByText(/or use a cloud link/i)).toBeNull();
411     expect(screen.getByTestId("upload-meta").textContent).toMatch(/1 MB of 2 MB uploaded
· 0:0\d left/);
412
413     await act(async () => {
414         releasePart();
415     });
416 });
417
418 it("offers Drive as a storage medium when the engine reports a usable mount", async ()
=> {
419     vi.stubGlobal(
420         "fetch",
421         fetchStore({
422             caps: {
423                 storage: {
424                     backends: [
425                         { id: "local", kind: "local", label: "This computer", usable: true },
426                         { id: "gdrive", kind: "gdrive", label: "Drive", usable: true },
427                     ],
428                 },
429             },
430         })),
431     );
432     render(<IngestPanel onLoaded={vi.fn()} />);
433     await screen.findByTestId("medium-picker");
434     fireEvent.click(screen.getByTestId("medium-drive"));
435     expect(screen.getByPlaceholderText("gdrive://Matches")).toBeTruthy();
436 });
437
438 it("stores an uploaded video into the chosen medium, then processes the stored copy",
async () => {
439     const assetBodies: Record<string, unknown>[] = [];
440     const processBodies: Record<string, unknown>[] = [];
441     vi.stubGlobal(
442         "fetch",
443         fetchStore({
444             caps: capsBucket,
445             onAssetBody: (b) => assetBodies.push(b),
446             onProcessBody: (b) => processBodies.push(b),
447             jobs: [{ job_id: "j", status: "done", video_id: "vidS", result: { status:
"success", video_id: "vidS" } }],
448         })),
449     );
450     render(<IngestPanel onLoaded={vi.fn()} />);
451     await screen.findByTestId("medium-picker");
452     fireEvent.click(screen.getByTestId("medium-bucket"));

```

```

453         fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gs://b/videos" } });
454         const file = new File([new Uint8Array([1, 2, 3])], "match.mp4", { type: "video/mp4"
});
455         const fileInput = document.querySelector('input[type="file"]') as HTMLInputElement;
456         await act(async () => {
457             fireEvent.change(fileInput, { target: { files: [file] } });
458         });
459         expect(screen.getByTestId("ingest-video-title").textContent).toContain("match.mp4");
460         expect(assetBodies).toHaveLength(0);
461         clickStartUpload();
462         // The upload ? createAsset ? process chain runs: the asset is stored MD5-keyed,
then the STORED
463         // uri (not the raw upload path) is what's indexed.
464         await waitFor(() => expect(assetBodies).toHaveLength(1));
465         expect(assetBodies[0]).toEqual({ uploaded_path: "/srv/up/match.mp4", medium_uri:
"gs://b/videos" });
466         await waitFor(() => expect(processBodies).toHaveLength(1));
467         expect(processBodies[0].video_path).toBe("gs://b/videos/vidS.mp4");
468     });
469
470     it("surfaces a store failure loudly (e.g. the engine's 507 no-space)", async () => {
471         const fetchFail = fetchStore({ caps: capsBucket });
472         // Override /api/assets to a 507.
473         const wrapped = vi.fn(async (url: string | URL, init?: RequestInit) => {
474             if (String(url) === "/api/assets") return httpOk(507, { detail: "Not enough free
disk space for store" });
475             return (fetchFail as unknown as (u: string | URL, i?: RequestInit) =>
Promise<unknown>)(url, init);
476         }) as unknown as FetchFn;
477         vi.stubGlobal("fetch", wrapped);
478         render(<IngestPanel onLoaded={vi.fn()} />);
479         await screen.findByTestId("medium-picker");
480         fireEvent.click(screen.getByTestId("medium-bucket"));
481         fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gs://b/videos" } });
482         const file = new File([new Uint8Array([1, 2, 3])], "match.mp4", { type: "video/mp4"
});
483         await act(async () => {
484             fireEvent.change(document.querySelector('input[type="file"]') as HTMLInputElement,
{ target: { files: [file] } });
485         });
486         clickStartUpload();
487         expect(await screen.findByTestId("ingest-error")).toBeTruthy();
488     });
489
490     it("gates cloud on a cost ack, then sends compute=cloud", async () => {
491         const bodies: Record<string, unknown>[] = [];
492         vi.stubGlobal(
493             "fetch",
494             fetchAll({
495                 caps: { deployment: { cloud_available: true } },
496                 onProcessBody: (b) => bodies.push(b),
497                 jobs: [{ job_id: "j", status: "done", video_id: "vidC", result: { status:
"success", video_id: "vidC" } }],
498             }),
499         );
500         render(<IngestPanel onLoaded={vi.fn()} />);
501         await screen.findByTestId("compute-picker");
502         // Type the URI while LOCAL (the input is enabled), then switch to CLOUD.
503         typeUri("gs://bucket/match.mp4");
504         fireEvent.click(screen.getByRole("button", { name: /in the cloud/i }));
505         // Cost note + ack appear; submit is gated until the cost is accepted.
506         expect(screen.getByTestId("compute-cost")).toBeTruthy();
507         expect(screen.getByTestId("compute-ack-required")).toBeTruthy();

```

```

508         const submitBtn = screen.getByRole("button", { name: /process video/i }) as
HTMLButtonElement;
509         expect(submitBtn.disabled).toBe(true);
510         clickProcess(); // a disabled button does nothing ? still no submit
511         expect(bodies).toHaveLength(0);
512         // Accept the cost ? the gate lifts ? run on the cloud.
513         fireEvent.click(screen.getByRole("checkbox"));
514         expect(submitBtn.disabled).toBe(false);
515         clickProcess();
516         await act(async () => {});
517         expect(bodies).toHaveLength(1);
518         expect(bodies[0].compute).toBe("cloud");
519     });
520
521     it("shows the upload-moment privacy disclosure + a link to the full policy on a cloud
server (#154)", async () => {
522         vi.stubGlobal("fetch", fetchAll({ caps: { deployment: { cloud_available: true } }
})));
523         render(<IngestPanel onLoaded={vi.fn()} />);
524         const box = await screen.findByTestId("upload-disclosure");
525         // Honest about WHERE footage goes (the blocker this closes), and no "never leaves
your device" lie.
526         expect(box.textContent).toMatch(/Google Cloud bucket in Singapore/);
527         expect(box.textContent).not.toMatch(/never leave/i);
528         const link = screen.getByRole("link", { name: /privacy policy/i }) as
HTMLAnchorElement;
529         expect(link.getAttribute("href")).toBe("https://khelsutra.guru/privacy");
530         expect(link.className).toMatch(/b(?:min-h-11|h-11)b/); // real tap target (A4 /
WCAG 2.5.5)
531     });
532
533     it("suppresses the disclosure on a local appliance where nothing is uploaded (#154)",
async () => {
534         vi.stubGlobal("fetch", fetchAll({ caps: {} })); // no deployment.cloud_available ?
local-only
535         render(<IngestPanel onLoaded={vi.fn()} />);
536         await screen.findByTestId("ingest-panel");
537         expect(screen.queryByTestId("upload-disclosure")).toBeNull();
538     });
539
540     // --- #161: the failed video is always named on the retry screen
-----
541     // Owner ask (issue #161 + comment): the rendered page must DETERMINISTICALLY show
which video the
542     // session is working on ? especially on the failure/retry screen, where the name was
being lost.
543
544     it("names the video on the retry screen when a single-shot upload fails (#161)", async
() => {
545         // Local medium, engine unreachable at upload/init ? the upload fails and the retry
card appears.
546         vi.stubGlobal(
547             "fetch",
548             vi.fn(async (url: string | URL) => {
549                 const u = String(url);
550                 if (u === "/api/capabilities") return httpOk(200, {});
551                 if (u === "/api/upload/init") throw new Error("offline");
552                 throw new Error(`unexpected fetch ${u}`);
553             }) as unknown as FetchFn,
554         );
555         render(<IngestPanel onLoaded={vi.fn()} />);
556         await screen.findByTestId("ingest-panel");
557         const file = new File([new Uint8Array([1, 2, 3])], "Quarterfinal.mp4", { type:
"video/mp4" });
558         await act(async () => {

```

```

559         fireEvent.change(document.querySelector('input[type="file"]') as HTMLInputElement,
{ target: { files: [file] } });
560     });
561     await act(async () => {
562         clickStartUpload();
563     });
564     // The retry card itself names the failed video (never a bare placeholder)...
565     const errorCard = await screen.findByTestId("ingest-error");
566     expect(errorCard.textContent).toContain("Quarterfinal.mp4");
567     // ...and the persistent title still shows it too.
568     expect(screen.getByTestId("ingest-video-
title").textContent).toContain("Quarterfinal.mp4");
569     });
570
571     it("names the video on the retry screen when a URI submission fails (#161)", async ()
=> {
572         vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" }, [])));
573         render(<IngestPanel onLoad={vi.fn()} />);
574         typeUri("gs://bucket/SemiFinal.mp4");
575         clickProcess();
576         const errorCard = await screen.findByTestId("ingest-error");
577         expect(errorCard.textContent).toContain("SemiFinal.mp4");
578     });
579
580     it("still names the video on the retry screen after a reload, once the resumed job
fails (#161)", async () => {
581         // The real gap: the page was reloaded mid-processing (component state gone), then
the resumed job
582         // fails. The name must survive via the restored job ? error phase ? retry card,
with NO prior
583         // in-page interaction. This fails on the old code (error phase dropped the display
name).
584         window.localStorage.setItem(
585             "kheIsutra.activeIngestJob",
586             JSON.stringify({ jobId: "jR", uri: "gs://b/reload.mp4", cid: "c1", startedAt:
1000, displayName: "reload.mp4" })),
587         );
588         window.history.replaceState(null, "", "?job=jR");
589         vi.stubGlobal(
590             "fetch",
591             vi.fn(async (url: string | URL) => {
592                 const u = String(url);
593                 if (u === "/api/capabilities") return httpOk(200, {});
594                 if (u.startsWith("/api/jobs/")) return httpOk(200, { job_id: "jR", status:
"failed", error: "worker crashed" });
595                 throw new Error(`unexpected fetch ${u}`);
596             }) as unknown as FetchFn,
597         );
598         render(<IngestPanel onLoad={vi.fn()} />);
599         const errorCard = await screen.findByTestId("ingest-error");
600         expect(errorCard.textContent).toContain("reload.mp4");
601     });
602 });
603

```

23. assistant (2026-07-07T15:10:33.947Z)

The SPA tests look solid ? line 169 (`cloud\_available:true`, no `compute\_target`) asserts `compute=local`, which guards against over-defaulting, and line 188 asserts the hosted case sends exactly one `compute:"cloud"`. Now let me verify the backend config ground truth for the untested `tracknet\_hybrid`/`fusion\_hybrid` branches, and confirm the real registered segmenter names.

24. user (2026-07-07T15:10:37.799Z)

```

=== indexing config: wasb / tracknet / fusion / substrate / weights_path ===
51: weights_path: str = ""
69: """Typed config for the live WASB shuttle substrate (indexing.wasb).
71: Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
79: conda_env: str = "wasb"
80: weights_path: str = (
81:     "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
93:     "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
115: # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8, no
140:     f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda' or "
149:     # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the flag
152:     raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got {v})")
163:     f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
168:class FusionConfig(BaseModel):
169:     """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
171:     Every default reproduces control behaviour: the fusion segmenter persists the
179:     # Which trajectory substrate seeds the windows (shuttle stays primary).
180:     substrate: Literal["wasb", "tracknet"] = "wasb"
181:     # Windowing velocity threshold for the fusion family (the engine reads
182:     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
183:     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
202:class IndexingConfig(BaseModel):
205:     # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
209:     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
285:     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff entirely
and
293:     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294:     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295:     fusion: FusionConfig = Field(default_factory=FusionConfig)

```

25. user (2026-07-07T15:10:41.437Z)

```

=== registered segmenter names (register/name= in segmenters) ===
backend/pipeline/segmenters/wasb_hybrid.py:21:from backend.pipeline.segmenters.base import
segmenter_registry
backend/pipeline/segmenters/wasb_hybrid.py:68:segmenter_registry.register(WasbHybridSegmenter)
backend/pipeline/segmenters/tracknet_hybrid.py:19:from backend.pipeline.segmenters.base import
segmenter_registry
backend/pipeline/segmenters/tracknet_hybrid.py:48:segmenter_registry.register(TrackNetHybridSegm
enter)
backend/pipeline/segmenters/fusion_hybrid.py:37:from backend.pipeline.segmenters.base import
segmenter_registry
backend/pipeline/segmenters/fusion_hybrid.py:191:         return {k: 0.0 for k in
ff.PERSON_FEATURE_NAMES}
backend/pipeline/segmenters/fusion_hybrid.py:243:segmenter_registry.register(FusionSegmenter)
=== grep the registry for these names anywhere ===
backend/pipeline/segmenters/fusion_hybrid.py:24:and hands off identically to `wasb_hybrid`.
Registered as ``fusion_hybrid``, NOT the default
backend/pipeline/segmenters/fusion_hybrid.py:58:         return "fusion_hybrid"
backend/pipeline/segmenters/tracknet_hybrid.py:29:         return "tracknet_hybrid"
backend/pipeline/segmenters/wasb_hybrid.py:31:         return "wasb_hybrid"

```

26. assistant (2026-07-07T15:11:00.489Z)

Let me confirm `TrackNetConfig.weights\_path` exists and its default, to pin down the untested branch precisely.

27. user (2026-07-07T15:11:02.013Z)

```

38         "this threshold for your recording device."
39     ),
40 )
41     relevance: ValidationRelevanceConfig = Field(

```

```

42         default_factory=ValidationRelevanceConfig
43     )
44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
63         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
64         # 0 = no extra floor (the headroom guard still applies).
65         min_free_gb: float = 0.0
66
67
68     class WasbIndexConfig(BaseModel):
69         """Typed config for the live WASB shuttle substrate (indexing.wasb).

```

28. user (2026-07-07T15:11:04.684Z)

```

158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
162     # "WASB weights not found". (Names are the registered segmenter identities; see
163     # backend/pipeline/segmenters.)
164     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
165     "fusion_hybrid"})
166
167     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
168         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
169         present + readable
170         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
171         ``indexing.<det>.weights_path``)
172         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
173         that would
174         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
175         default / TrackNet).
176         A non-weights segmenter returns True (nothing local to check)."""
177         name = (seg_name or "").strip().lower()
178         if name not in _WEIGHTS_BACKED_SEGMENTERS:
179             return True
180         idx = getattr(cfg, "indexing", None)
181         substrate = (
182             getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
183         ).strip().lower()
184         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==

```

```

"tracknet"):
180     path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181     else: # wasb_hybrid, or fusion on the (default) WASB substrate
182         path = (
183             os.environ.get("WASB_WEIGHTS")
184             or getattr(getattr(idx, "wasb", None), "weights_path", "")
185             or ""
186         )
187     return bool(path) and os.path.exists(os.path.expanduser(path))

```

29. assistant (2026-07-07T15:12:50.500Z)

I now have all the ground truth I need. Let me consolidate my analysis.

```

**Verified facts:**
- Registered segmenter identities are exactly `wasb_hybrid`, `tracknet_hybrid`, `fusion_hybrid`
(match `WEIGHTS_BACKED_SEGMENTERS`).
- `_local_detector_weights_present` (orchestration_new.py:166-187) has a non-trivial branch:
`tracknet_hybrid` OR (`fusion_hybrid` + `substrate=="tracknet"`) ? reads
`cfg.indexing.tracknet.weights_path`; else reads
`WASB_WEIGHTS`/`cfg.indexing.wasb.weights_path`. `TrackNetConfig.weights_path` defaults to `""`;
`FusionConfig.substrate` is `Literal["wasb","tracknet"]="wasb"`.
- `_failed(msg, False)` yields `{path:"not_run", requested:req.compute, target:"cloud_run",
dispatched:False}` ? so the fail-fast test's provenance assertions are correct.
- `_cloud_available` is True via `compute_target=="cloud_run"` OR via
`CLOUD_RUN_JOB`+`GOOGLE_CLOUD_PROJECT`+`storage.output_root` env.

```

```

**Backend guardrail tests** all use `wasb_hybrid` (weights-backed) or `motion` (non-weights).
The fail-fast test asserts the exact message, `dispatched=False`, `path="not_run"`,
`requested="local"`, and `fake.calls==[]`; the escape-hatch test is real (real on-disk weights ?
`in_process`). Good pair, not tautological.

```

```

**SPA tests:** line 169 (`cloud_available:true`, no `compute_target`) asserts `compute=local` ?
this genuinely guards against over-defaulting; line 188 asserts exactly one body with
`compute:"cloud"`. Adequate.

```

```

**Two real gaps:** (1) `tracknet_hybrid`/`fusion_hybrid` ? the whole branchy half of the helper
(tracknet weights path + fusion substrate switch) ? have zero coverage though the fix declares
all three in scope; (2) the env-wired `_cloud_available` route into the guardrail is never
exercised.

```

30. user (2026-07-07T15:13:06.811Z)

Structured output provided successfully